

Sudoku Generator Design Document

Program Overview

This project is a Java console application that generates and displays a fully solved 9×9 Sudoku puzzle. The goal of the program is to demonstrate programming concepts like 2D arrays, ArrayLists, methods, and algorithmic design to solve problems.

A Sudoku puzzle is a logic-based grid that follows strict rules:

- Each row must contain the numbers 1 through 9 exactly once
- Each column must contain the numbers 1 through 9 exactly once
- Each 3×3 subgrid must contain the numbers 1 through 9 exactly once

The program generates a valid Sudoku solution without hard-coding values. Instead, it uses a mathematical pattern to build the board. This ensures it is correct while still allowing variation through randomness. The final board is printed in a clean console format for easy readability.

Data Structures

a. 2D Array

The main structure used is a 9×9 integer array:

```
int[][] board = new int[9][9];
```

This 2D array represents the Sudoku grid, where each element corresponds to a specific cell identified by row and column indices. The structure allows the program to efficiently organize and access all values in a grid format similar to a real Sudoku puzzle.

Nested loops are used to fill and display the board. The 2D array is essential because it supports systematic traversal of rows, columns, and subgrids, which is necessary to maintain Sudoku rules.

b. ArrayList

The program also uses an ArrayList<Integer> to store values 1 through 9:

```
ArrayList<Integer> base = new ArrayList<>();
```

This list is shuffled using Collections.shuffle() to introduce randomness into the starting configuration. The shuffled values are then used as the foundation for generating the Sudoku board.

The ArrayList provides flexibility because it allows easy reordering of values and supports built-in methods like shuffle, which a standard array does not. This helps ensure that each generated Sudoku board is different while still valid.

Algorithm Design

The program uses a pattern-based algorithm instead of random placement, which ensures the Sudoku rules are always satisfied.

Steps:

1. Create an ArrayList with numbers 1–9
2. Shuffle the list for randomness
3. Use nested loops to fill the 2D array
4. Apply a mathematical formula to determine placement

Formula Used:

$(col + (row * 3) + (row / 3)) \% 9$

Explanation:

- col controls horizontal movement
- row * 3 shifts each row's starting position
- row / 3 adjusts the pattern every 3 rows for correct subgrid structure
- % 9 keeps values within range

Why It Works:

This formula creates a controlled shifting pattern so that:

- Each row contains numbers 1–9 without repetition
- Each column remains unique
- Each 3×3 subgrid stays valid

Because the base list is shuffled first, each run produces a different valid Sudoku solution while maintaining structural correctness.

Methods

main()

The entry point of the program. It creates the Sudoku object and calls methods to generate and print the board in the correct order.

generateBoard()

This method builds the Sudoku board by:

- Creating and shuffling the ArrayList
- Filling the 2D array using nested loops
- Applying the mathematical pattern formula

It is the core logic of the program and ensures the board is valid.

printBoard()

This method displays the Sudoku board in a readable format. It:

- Prints rows and columns
- Adds vertical separators for 3×3 grids
- Adds horizontal lines every 3 rows

This makes the output visually similar to a real Sudoku puzzle.

Testing Plan

Test 1: Multiple Runs

The program was executed multiple times to ensure:

- Each board is different due to shuffling
- Each board remains valid

Result: Boards changed each run while staying correct.

Test 2: Row Validation

Each row was checked to confirm it contains numbers 1–9 without repetition.

Result: All rows were valid.

Test 3: Column Validation

Each column was checked for duplicates.

Result: All columns were valid.

Test 4: Subgrid Validation

Each 3×3 grid was checked to ensure it contains all numbers 1–9 exactly once.

Result: All subgrids were valid.

Challenges

Challenge 1: Ensuring Valid Sudoku Rules

One major challenge was preventing duplicate numbers in rows, columns, and 3×3 subgrids. Early attempts using random placement resulted in invalid boards. This was solved by switching to a structured pattern-based algorithm instead of random assignment.

Challenge 2: Understanding the Pattern Formula

The formula used to generate the board was initially difficult to understand and implement correctly. Small mistakes caused incorrect board layouts. To solve this, the formula was tested step-by-step and debugged using intermediate outputs.